- BÁO CÁO CUỐI KỲ
- MYLOGIC EDA TOOL
  - Unified Electronic Design Automation Tool with Advanced VLSI CAD Algorithms
  - MỤC LỤC
- PHẦN 1: MỞ ĐẦU
  - 1.1. Giới Thiệu
    - Đặt Vấn Đề
    - Lý Do Chọn Đề Tài
    - Mục Tiêu Nghiên Cứu
    - Đối Tượng Nghiên Cứu
    - Phạm Vi Nghiên Cứu
  - 1.2. Cấu Trúc Báo Cáo
- PHẦN 2: TỔNG QUAN VỀ EDA VÀ LOGIC SYNTHESIS
  - 2.1. Electronic Design Automation (EDA)
    - 2.1.1. Khái Niệm
    - 2.1.2. Vai Trò của EDA trong VLSI Design
    - 2.1.3. Design Flow
  - 2.2. Logic Synthesis
    - 2.2.1. Định Nghĩa
    - 2.2.2. Quy Trình Synthesis
    - 2.2.3. Synthesis Goals
  - 2.3. And-Inverter Graph (AIG)
    - 2.3.1. Định Nghĩa
    - 2.3.2. Cấu Trúc AIG
    - 2.3.3. Tại Sao Chỉ Dùng AND và NOT?
    - 2.3.4. Biểu Diễn NOT trong AIG
    - 2.3.5. Conversion Các Cổng Cơ Bản sang AIG
      - AND
      - OR (De Morgan's Law)
      - XOR
      - NAND
      - NOR
      - XNOR
  - 2.4. Related Work
    - 2.4.1. Các Công Cụ EDA Hiện Có
    - 2.4.2. So Sánh với MyLogic EDA Tool

- PHẦN 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG
  - 3.1. Yêu Cầu Hệ Thống
    - 3.1.1. Yêu Cầu Chức Năng
    - 3.1.2. Yêu Cầu Phi Chức Năng
    - 3.1.3. Yêu Cầu Kỹ Thuật
  - 3.2. Kiến Trúc Hệ Thống
    - 3.2.1. Tổng Quan Kiến Trúc
    - 3.2.2. Các Module Chính
  - 3.3. Quy Trình Xử Lý
    - 3.3.1. Complete Flow
    - 3.3.2. Data Flow
  - 3.4. Cấu Trúc Dữ Liệu
    - 3.4.1. AIGNode
    - 3.4.2. MultiBitAIGNode
- PHẦN 4: VERILOG PARSER
  - 4.1. Tổng Quan
    - 4.1.1. Pipeline Xử Lý
  - 4.2. Tokenization (Lexical Analysis)
    - 4.2.1. Mục Đích
    - 4.2.2. Token Types
    - 4.2.3. Code Cleaning
  - 4.3. Syntax Analysis
    - 4.3.1. Module Parsing
    - 4.3.2. Port Declarations
    - 4.3.3. Parameters và Localparams
  - 4.4. Expression Parsing
    - 4.4.1. Recursive Descent Parsing
    - 4.4.2. Operator Precedence
    - 4.4.3. Parentheses Matching
    - 4.4.4. Nested NOT Expressions
  - 4.5. Supported Constructs
    - 4.5.1. Assign Statements
    - 4.5.2. Always Blocks
    - 4.5.3. Case Statements
    - 4.5.4. Generate Blocks
    - 4.5.5. Bit Manipulation
    - 4.5.6. Module Instantiation
  - 4.6. AST (Abstract Syntax Tree)

        - 4.6.1. AST Structure
        - 4.6.2. AST Dump Utility
- PHẦN 5: SYNTHESIS ENGINE - NETLIST TO AIG CONVERSION
    - 5.1. Tổng Quan Synthesis
        - 5.1.1. Mục Đích
        - 5.1.2. Phân Biệt Synthesis vs Optimization
    - 5.2. AIG Data Structure
        - 5.2.1. AIG Class
        - 5.2.2. AIGNode Structure
    - 5.3. Gate Conversion Algorithms
        - 5.3.1. AND Gate
        - 5.3.2. OR Gate (De Morgan's Law)
        - 5.3.3. XOR Gate
        - 5.3.4. NAND Gate
        - 5.3.5. NOR Gate
        - 5.3.6. XNOR Gate
        - 5.3.7. NOT Gate
    - 5.4. Conversion Algorithm
        - 5.4.1. Algorithm Chi Tiết
        - 5.4.2. Signal Mapping
        - 5.4.3. Topological Ordering
    - 5.5. Multi-bit Operations
        - 5.5.1. MultiBitAIGNode
        - 5.5.2. ADD Operation
        - 5.5.3. SUB Operation
        - 5.5.4. MUX Operation
        - 5.5.5. EQ Operation
    - 5.6. Structural Hashing
        - 5.6.1. Hash Key
        - 5.6.2. Constant Folding
        - 5.6.3. Duplicate Elimination
- PHẦN 6: OPTIMIZATION ALGORITHMS
    - 6.1. Tổng Quan Optimization
        - 6.1.1. Mục Đích
        - 6.1.2. Optimization Levels
    - 6.2. Structural Hashing (Strash)
        - 6.2.1. Định Nghĩa
        - 6.2.2. Implementation

        - 6.2.3. Benefits
    - 6.3. Dead Code Elimination (DCE)
        - 6.3.1. Định Nghĩa
        - 6.3.2. Algorithm
        - 6.3.3. Levels
    - 6.4. Common Subexpression Elimination (CSE)
        - 6.4.1. Định Nghĩa
        - 6.4.2. Algorithm
        - 6.4.3. Implementation
    - 6.5. Constant Propagation (ConstProp)
        - 6.5.1. Định Nghĩa
        - 6.5.2. Algorithm
        - 6.5.3. Benefits
    - 6.6. Logic Balancing (Balance)
        - 6.6.1. Định Nghĩa
        - 6.6.2. Algorithm
        - 6.6.3. Benefits
- PHẦN 7: TECHNOLOGY MAPPING
    - 7.1. Tổng Quan Technology Mapping
        - 7.1.1. Mục Đích
        - 7.1.2. Strategies
    - 7.2. Technology Libraries
        - 7.2.1. ASIC Libraries
        - 7.2.2. FPGA Libraries
    - 7.3. Mapping Algorithm
        - 7.3.1. Pattern Matching
        - 7.3.2. Output Generation
- PHẦN 8: VERIFICATION
    - 8.1. Tổng Quan Verification
        - 8.1.1. Mục Đích
        - 8.1.2. Verification Types
    - 8.2. Verification Methodology
        - 8.2.1. Industry Standard
        - 8.2.2. Test Vector Generation
    - 8.3. ModelSim Integration
        - 8.3.1. Integration Flow
        - 8.3.2. Verilog Generation
        - 8.3.3. Testbench Generation

- 8.4. Verification Results
  - 8.4.1. Result Format
  - 8.4.2. Usage
- PHẦN 9: VLSI CAD ALGORITHMS
  - 9.1. Binary Decision Diagrams (BDD)
    - 9.1.1. Định Nghĩa
    - 9.1.2. Applications
  - 9.2. Boolean Expression Diagrams (BED)
    - 9.2.1. Enhanced BDD
  - 9.3. SAT Solver
    - 9.3.1. Boolean Satisfiability
    - 9.3.2. Applications
  - 9.4. Placement Algorithms
    - 9.4.1. Random Placement
    - 9.4.2. Force-Directed Placement
    - 9.4.3. Simulated Annealing
  - 9.5. Routing Algorithms
    - 9.5.1. Maze Routing (Lee's Algorithm)
    - 9.5.2. Rip-up and Reroute
  - 9.6. Static Timing Analysis (STA)
    - 9.6.1. Critical Path Analysis
    - 9.6.2. Slack Calculation
- PHẦN 10: SIMULATION
  - 10.1. Scalar Simulation
    - 10.1.1. Single-bit Logic Gates
  - 10.2. Vector Simulation
    - 10.2.1. Multi-bit Operations
  - 10.3. Auto-detection
- PHẦN 11: CLI INTERFACE
  - 11.1. Interactive Shell
    - 11.1.1. Features
    - 11.1.2. Commands
- PHẦN 12: IMPLEMENTATION DETAILS
  - 12.1. Programming Language và Tools
  - 12.2. Code Organization
  - 12.3. Key Design Decisions
- PHẦN 13: KẾT QUẢ VÀ ĐÁNH GIÁ
  - 13.1. Test Cases

        - 13.1.1. Test Suite
        - 13.1.2. Test Results
    - 13.2. Limitations
        - 13.2.1. Current Limitations
        - 13.2.2. Workarounds
    - 13.3. Comparison với Yosys
- PHẦN 14: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN
    - 14.1. Kết Luận
        - 14.1.1. Tổng Kết
        - 14.1.2. Đóng Góp
    - 14.2. Hạn Chế
    - 14.3. Hướng Phát Triển
        - 14.3.1. Short-term
        - 14.3.2. Long-term
- PHẦN 15: TÀI LIỆU THAM KHẢO
    - 15.1. Papers
    - 15.2. Books
    - 15.3. Standards
    - 15.4. Tools và Documentation
- PHỤ LỤC
    - Phụ Lục A: Code Examples
        - A.1. Example Verilog File
        - A.2. Usage Example
    - Phụ Lục B: Command Reference
    - Phụ Lục C: API Documentation

BÁO CÁO CUỐI KỲ

MYLOGIC EDA TOOL

Unified Electronic Design Automation Tool with Advanced VLSI CAD Algorithms

Tác giả: MyLogic EDA Tool Team **Phiên bản:** 2.0.0 **Năm:** 2024

MỤC LỤC

1. Mở Đầu
2. Tổng Quan Về EDA và Logic Synthesis
3. Phân Tích và Thiết Kế Hệ Thống
4. Verilog Parser
5. Synthesis Engine - Netlist to AIG Conversion
6. Optimization Algorithms
7. Technology Mapping
8. Verification
9. VLSI CAD Algorithms
10. Simulation
11. CLI Interface
12. Implementation Details
13. Kết Quả và Đánh Giá
14. Kết Luận và Hướng Phát Triển
15. Tài Liệu Tham Khảo

PHẦN 1: MỞ ĐẦU

1.1. Giới Thiệu

Đặt Vấn Đề

Trong thiết kế VLSI (Very Large Scale Integration), Electronic Design Automation (EDA) đóng vai trò quan trọng trong việc chuyển đổi mô tả logic mức cao (RTL - Register Transfer Level) sang các netlist mức gate phù hợp với công nghệ sản xuất. Logic synthesis là một bước then chốt trong quy trình EDA, chuyển đổi RTL description thành gate-level netlist thông qua các bước optimization và technology mapping.

Hiện tại, các công cụ EDA chuyên nghiệp như Synopsys Design Compiler, Cadence Genus đều là các phần mềm thương mại với chi phí cao, không phù hợp cho mục đích giáo dục và nghiên cứu. Các công cụ mã nguồn mở như Yosys, ABC mặc dù mạnh mẽ nhưng có độ phức tạp cao và khó tùy chỉnh.

Lý Do Chọn Đề Tài

Việc phát triển một công cụ EDA đơn giản, dễ hiểu nhưng đầy đủ tính năng sẽ giúp:

- **Học sinh, sinh viên**: Hiểu rõ hơn về quy trình logic synthesis
- **Nghiên cứu viên**: Có thể tùy chỉnh và thử nghiệm các thuật toán mới
- **Giảng viên**: Có công cụ để minh họa các khái niệm EDA

Mục Tiêu Nghiên Cứu

Dự án MyLogic EDA Tool nhằm mục tiêu:

1. **Phát triển Verilog Parser**: Xây dựng parser có khả năng parse các cấu trúc Verilog cơ bản và phức tạp, bao gồm module declarations, always blocks, case statements, generate blocks, và các operations khác nhau.
2. **Xây dựng Synthesis Engine**: Implement synthesis engine chuyển đổi netlist sang And-Inverter Graph (AIG) - một canonical form hiệu quả cho logic synthesis.
3. **Triển Khai Optimization Algorithms**: Implement các thuật toán optimization quan trọng như Structural Hashing, Dead Code Elimination, Common Subexpression Elimination, Constant Propagation, và Logic Balancing.
4. **Technology Mapping**: Xây dựng technology mapping engine có khả năng map AIG sang các technology libraries khác nhau (ASIC, FPGA).
5. **Verification System**: Phát triển hệ thống verification để đảm bảo tính đúng đắn của synthesis và optimization bằng cách so sánh kết quả simulation.
6. **VLSI CAD Algorithms**: Implement các thuật toán VLSI CAD cơ bản như BDD, SAT Solver, Placement, Routing, và Timing Analysis.

Đối Tượng Nghiên Cứu

- Verilog HDL parsing và AST generation
- Logic synthesis: Netlist → AIG conversion
- AIG optimization algorithms

- Technology mapping
- Functional verification

Phạm Vi Nghiên Cứu

Dự án tập trung vào:

- **Combinational logic synthesis**: Hỗ trợ các cổng combinational (AND, OR, XOR, NAND, NOR, XNOR, NOT)
- **Multi-bit operations**: ADD, SUB, MUX, EQ (đã implement)
- **Optimization**: Các thuật toán optimization cơ bản và nâng cao
- **Verification**: Functional verification cho combinational logic

Giới hạn hiện tại:

- Sequential logic (flip-flops, registers) chưa được hỗ trợ đầy đủ
- Memory arrays cần được xử lý riêng
- Complex always blocks cần được cải thiện

1.2. Cấu Trúc Báo Cáo

Báo cáo được chia thành các phần chính:

- **Phần 2**: Tổng quan về EDA và Logic Synthesis - Giới thiệu các khái niệm cơ bản
- **Phần 3**: Phân tích và Thiết kế Hệ thống - Kiến trúc và thiết kế tổng thể
- **Phần 4**: Verilog Parser - Chi tiết implementation của parser
- **Phần 5**: Synthesis Engine - Netlist to AIG conversion
- **Phần 6**: Optimization Algorithms - Các thuật toán tối ưu hóa
- **Phần 7**: Technology Mapping - Mapping sang target libraries
- **Phần 8**: Verification - Hệ thống verification
- **Phần 9**: VLSI CAD Algorithms - Các thuật toán VLSI CAD
- **Phần 10-14**: Simulation, CLI, Implementation, Kết quả, Kết luận

---

PHẦN 2: TỔNG QUAN VỀ EDA VÀ LOGIC SYNTHESIS

2.1. Electronic Design Automation (EDA)

2.1.1. Khái Niệm

Electronic Design Automation (EDA) là một tập hợp các công cụ phần mềm được sử dụng để thiết kế các hệ thống điện tử như vi mạch tích hợp (Integrated Circuits - ICs) và bảng mạch in (Printed Circuit Boards - PCBs).

EDA tools tự động hóa các tác vụ thiết kế phức tạp, giúp các kỹ sư:

- Thiết kế và kiểm tra các hệ thống điện tử
- Mô phỏng và xác minh thiết kế
- Tối ưu hóa về diện tích, hiệu năng, và công suất tiêu thụ
- Tạo ra các file cần thiết cho quá trình sản xuất

2.1.2. Vai Trò của EDA trong VLSI Design

Trong thiết kế VLSI, EDA đóng vai trò then chốt trong:

1. **Design Entry**: Nhập thiết kế bằng HDL (Verilog, VHDL)
2. **Simulation**: Mô phỏng để kiểm tra tính đúng đắn
3. **Synthesis**: Chuyển đổi RTL sang gate-level netlist
4. **Place & Route**: Sắp xếp và định tuyến các cells
5. **Verification**: Xác minh tính đúng đắn ở các mức khác nhau
6. **Timing Analysis**: Phân tích timing constraints

2.1.3. Design Flow

Quy trình thiết kế VLSI điển hình:

```
RTL Description (Verilog)
    ↓
Functional Simulation
    ↓
Logic Synthesis
    ↓
Gate-level Netlist
    ↓
Technology Mapping
```

```
    ↓
Place & Route
    ↓
Timing Analysis
    ↓
GDSII (Manufacturing)
```

2.2. Logic Synthesis

2.2.1. Định Nghĩa

Logic Synthesis là quá trình chuyển đổi mô tả logic mức cao (high-level description) như RTL code sang một representation mức thấp hơn (gate-level netlist) phù hợp với công nghệ sản xuất chip.

2.2.2. Quy Trình Synthesis

Logic synthesis bao gồm các bước chính:

1. **Parsing & Elaboration**
    - Parse HDL code
    - Resolve hierarchy
    - Expand parameters và generate blocks
2. **Technology-Independent Optimization**
    - Convert sang canonical form (AIG, BDD)
    - Apply optimization algorithms
    - Logic minimization
3. **Technology Mapping**
    - Map logic functions sang target library
    - Optimize cho area/delay/power
4. **Optimization**
    - Post-mapping optimization
    - Timing optimization

2.2.3. Synthesis Goals

Mục tiêu của synthesis:

- **Functional Correctness**: Đảm bảo output giống với input
- **Area Optimization**: Giảm số lượng gates
- **Delay Optimization**: Tối ưu hóa critical path
- **Power Optimization**: Giảm công suất tiêu thụ

2.3. And-Inverter Graph (AIG)

2.3.1. Định Nghĩa

And-Inverter Graph (AIG) là một directed acyclic graph (DAG) biểu diễn Boolean functions chỉ sử dụng:

- **AND gates** (2-input AND)
- **Inverters** (NOT gates, được biểu diễn bằng inversion flags)

2.3.2. Cấu Trúc AIG

Một AIG node có cấu trúc:

```
AIGNode {
    node_id: int              // Unique identifier
    node_type: str            // CONST0, CONST1, PI, AND
    left: AIGNode             // Left child (for AND nodes)
    right: AIGNode            // Right child (for AND nodes)
    left_inverted: bool       // Left input inversion flag
    right_inverted: bool      // Right input inversion flag
    var_name: str             // Variable name (for PI nodes)
}
```

2.3.3. Tại Sao Chỉ Dùng AND và NOT?

Functional Completeness: Bất kỳ Boolean function nào cũng có thể được biểu diễn chỉ bằng AND và NOT:

- **NOT**: `NOT(a) = !a`
- **OR**: `OR(a, b) = !(!a AND !b)` (De Morgan's law)
- **XOR**: `XOR(a, b) = (!a AND b) OR (a AND !b)`
- **NAND**: `NAND(a, b) = !(a AND b)`
- **NOR**: `NOR(a, b) = !(a OR b) = !a AND !b`

Lợi Ích:

1. **Canonical Form**: Dễ dàng so sánh và optimize
2. **Efficient Operations**: Các thao tác trên AIG rất hiệu quả
3. **Technology Independence**: Không phụ thuộc vào technology library cụ thể
4. **Structural Hashing**: Dễ dàng implement structural hashing để loại bỏ duplicates

2.3.4. Biểu Diễn NOT trong AIG

Trong AIG, NOT được biểu diễn bằng inversion flags thay vì explicit NOT nodes:

```
NOT(x) = AND(x, const1) với left_inverted=True
```

Điều này giúp:

- Giảm số lượng nodes
- Dễ dàng combine inversions
- Efficient structural hashing

2.3.5. Conversion Các Cổng Cơ Bản sang AIG

AND

```
AND(a, b) = AND(a, b)  // Giữ nguyên
```

OR (De Morgan's Law)

```
OR(a, b) = NOT(AND(NOT(a), NOT(b)))
         = AND(AND(NOT(a), NOT(b)), const1) với output inverted
```

XOR

```
XOR(a, b) = (!a AND b) OR (a AND !b)
          = NOT(AND(NOT(AND(NOT(a), b)), NOT(AND(a, NOT(b)))))
```

NAND

```
NAND(a, b) = NOT(AND(a, b))
           = AND(AND(a, b), const1) với output inverted
```

NOR

```
NOR(a, b) = NOT(OR(a, b))
          = AND(NOT(a), NOT(b))
          = AND(AND(a, const1), AND(b, const1)) với both inputs inverted
```

XNOR

```
XNOR(a, b) = NOT(XOR(a, b))
           = NOT(XOR(a, b))
```

2.4. Related Work

2.4.1. Các Công Cụ EDA Hiện Có

Commercial Tools:

- **Synopsys Design Compiler**: Industry standard synthesis tool
- **Cadence Genus**: Advanced synthesis với AI-based optimization
- **Mentor Graphics Precision**: RTL synthesis solution

Open Source Tools:

- **Yosys**: Open synthesis suite, được sử dụng rộng rãi

- **ABC**: AIG-based synthesis và verification tool
- **OpenROAD**: Complete RTL-to-GDSII flow

2.4.2. So Sánh với MyLogic EDA Tool

Feature	MyLogic	Yosys	ABC
Verilog Parser	✅ Custom	✅ Full support	❌
AIG Synthesis	✅	✅	✅
Optimization	✅ Basic	✅ Advanced	✅ Advanced
Technology Mapping	✅	✅	✅
Verification	✅ ModelSim	✅	✅
Educational Focus	✅	❌	❌
Code Readability	✅ High	⚠️ Medium	⚠️ Medium

Điểm Mạnh của MyLogic:

- Code dễ đọc và hiểu, phù hợp cho giáo dục
- Có documentation đầy đủ
- Modular design, dễ mở rộng
- Verification tích hợp

Hạn Chế:

- Chưa hỗ trợ đầy đủ các tính năng như Yosys/ABC
- Optimization chưa mạnh bằng các tools chuyên nghiệp
- Sequential logic chưa được hỗ trợ đầy đủ

PHẦN 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu Cầu Hệ Thống

3.1.1. Yêu Cầu Chức Năng

1. **Verilog Parsing**
    - Parse module declarations
    - Parse port declarations (input/output/inout)
    - Parse parameters và localparams
    - Parse assign statements
    - Parse always blocks (combinational và sequential)
    - Parse case statements
    - Parse generate blocks
    - Parse module instantiations
2. **Synthesis**
    - Convert netlist sang AIG
    - Hỗ trợ các cổng combinational
    - Hỗ trợ multi-bit operations (ADD, SUB, MUX, EQ)
3. **Optimization**
    - Structural Hashing (Strash)
    - Dead Code Elimination (DCE)
    - Common Subexpression Elimination (CSE)
    - Constant Propagation (ConstProp)
    - Logic Balancing (Balance)
4. **Technology Mapping**
    - Map AIG sang target library
    - Hỗ trợ ASIC và FPGA libraries
    - Area/delay/balanced optimization
5. **Verification**
    - Functional verification với ModelSim
    - Pre-synthesis vs Post-synthesis verification
    - Post-synthesis vs Post-optimization verification
6. **Simulation**

- Scalar simulation (single-bit)
- Vector simulation (multi-bit)
- Auto-detection mode

3.1.2. Yêu Cầu Phi Chức Năng

1. **Performance**: Xử lý được các design có kích thước trung bình
2. **Usability**: CLI interface thân thiện, dễ sử dụng
3. **Maintainability**: Code dễ đọc, modular, có documentation
4. **Extensibility**: Dễ dàng thêm features mới

3.1.3. Yêu Cầu Kỹ Thuật

- **Programming Language**: Python 3.8+
- **Dependencies**: NumPy, Matplotlib
- **Optional**: Yosys, ModelSim, Graphviz

3.2. Kiến Trúc Hệ Thống

3.2.1. Tổng Quan Kiến Trúc

MyLogic EDA Tool được thiết kế theo kiến trúc modular với các module chính:

```
┌──────────────────────────────────────────┐
│         CLI Interface (vector_shell)     │
└──────────────────────────────────────────┘
                    │
          ┌─────────┴─────────┐
          │                   │
┌─────────▼─────────┐   ┌─────────▼─────────┐
│ Verilog Parser    │   │  Complete Flow    │
└───────────────────┘   └───────────────────┘
          │                   │
          │         ┌─────────┴─────────┐
          │         │                   │
┌─────────▼─────────▼─────┐   ┌──────────▼──────────┐
│   Synthesis Engine      │   │  Optimization Flow  │
│   (Netlist → AIG)       │   │  (AIG Optimization) │
└─────────────────────────┘   └─────────────────────┘
          │                             │
          └─────────────────────────────┘
```

```
              │
              ▼
        ┌──────────────┐
        │ Technology Map  │
        │ (AIG → Library) │
        └──────────────┘
```

3.2.2. Các Module Chính

1. **Frontend Parsers** (`frontends/verilog/`)
    - Verilog parser với modular structure
    - Expression parser
    - Operation parsers (arithmetic, bitwise, logical, etc.)
2. **Core Synthesis** (`core/synthesis/`)
    - AIG data structure
    - Netlist to AIG converter
    - Multi-bit AIG support
3. **Optimization** (`core/optimization/`)
    - Structural Hashing
    - Dead Code Elimination
    - Common Subexpression Elimination
    - Constant Propagation
    - Logic Balancing
4. **Technology Mapping** (`core/technology_mapping/`)
    - Library loader
    - Technology mapping algorithms
5. **Verification** (`core/verification.py`)
    - Functional verification
    - ModelSim integration
6. **VLSI CAD** (`core/vlsi_cad/`)
    - BDD, BED
    - SAT Solver

    - Placement, Routing
    - Timing Analysis

7. **Simulation** (`core/simulation/`)
    - Scalar simulation
    - Vector simulation

8. **CLI Interface** (`cli/vector_shell.py`)
    - Interactive shell
    - 36 commands (bao gồm file operations, synthesis, optimization, verification, VLSI CAD)

3.3. Quy Trình Xử Lý

3.3.1. Complete Flow

```
Verilog File
    ↓
Parsing → Netlist Dictionary
    ↓
Synthesis → AIG
    ↓
Optimization → Optimized AIG
    ↓
Technology Mapping → Mapped Netlist
    ↓
Verification (Optional)
    ↓
Output (Verilog/JSON)
```

3.3.2. Data Flow

Netlist Dictionary Format:

```
{
    'name': 'module_name',
    'inputs': ['a', 'b'],
    'outputs': ['out'],
    'nodes': [
```

```
        {
            'id': 'and_0',
            'type': 'AND',
            'fanins': [['a', False], ['b', False]]
        }
    ],
    'attrs': {
        'output_mapping': {'out': 'and_0'}
    }
}
```

AIG Structure:

- Primary Inputs (PIs): Dictionary mapping signal names to AIG nodes
- Primary Outputs (POs): List of (node, inverted) tuples
- Nodes: Dictionary mapping node_id to AIGNode objects
- Hash Table: For structural hashing

3.4. Cấu Trúc Dữ Liệu

3.4.1. AIGNode

```
class AIGNode:
    def __init__(self, node_id, node_type, left=None, right=None,
                 left_inverted=False, right_inverted=False, var_name=None):
        self.node_id = node_id
        self.node_type = node_type  # CONST0, CONST1, PI, AND
        self.left = left
        self.right = right
        self.left_inverted = left_inverted
        self.right_inverted = right_inverted
        self.var_name = var_name
        self.level = 0
```

3.4.2. MultiBitAIGNode

```
class MultiBitAIGNode:
    def __init__(self, width: int, bits: List[AIGNode]):
        self.width = width
        self.bits = bits  # [bit0 (LSB), bit1, ..., bitN-1 (MSB)]
```

PHẦN 4: VERILOG PARSER

4.1. Tổng Quan

Verilog Parser là module đầu tiên trong pipeline, có nhiệm vụ chuyển đổi Verilog HDL code sang Netlist Dictionary - một representation trung gian có cấu trúc rõ ràng và dễ xử lý.

4.1.1. Pipeline Xử Lý

```
Verilog Source Code
    ↓
Tokenizer (Lexical Analysis)
    ↓
Syntax Analysis (Parser)
    ↓
AST (Abstract Syntax Tree) - Internal
    ↓
Netlist Dictionary
```

4.2. Tokenization (Lexical Analysis)

4.2.1. Mục Đích

Tokenization là bước đầu tiên, phân tích source code thành các tokens cơ bản.

4.2.2. Token Types

- **Keywords**: `module`, `input`, `output`, `wire`, `assign`, `endmodule`, `always`, `case`, etc.
- **Operators**:
  - Bitwise: `&`, `|`, `^`, `~`, `~&`, `~|`, `~^`, `^~`
  - Logical: `&&`, `||`, `!`
  - Arithmetic: `+`, `-`, `*`, `/`, `%`

    - Comparison: `==`, `!=`, `<`, `>`, `<=`, `>=`
    - Shift: `<<`, `>>`, `<<<`, `>>>`
- **Identifiers**: Tên biến, tên module (chữ cái, số, underscore)
- **Literals**: Số (decimal, hex, binary), strings
- **Delimiters**: `(`, `)`, `[`, `]`, `{`, `}`, `;`, `,`, `.`

4.2.3. Code Cleaning

Trước khi tokenize, code được làm sạch:

- Loại bỏ comments (`//`, `/* */`)
- Normalize whitespace
- Handle line continuations

4.3. Syntax Analysis

4.3.1. Module Parsing

Parser bắt đầu bằng việc tìm module declaration:

```
module module_name(
    input a,
    output b
);
    // module body
endmodule
```

Process:

1. Extract module name
2. Parse port list
3. Extract module body
4. Validate structure

4.3.2. Port Declarations

Parser hỗ trợ:

- **Input ports**: `input [7:0] data;`
- **Output ports**: `output [3:0] result;`
- **Inout ports**: `inout wire;`
- **Vector ports**: `[msb:lsb]` format
- **Signed/unsigned**: `signed` keyword

4.3.3. Parameters và Localparams

```
parameter WIDTH = 8;
parameter DEPTH = 16, ADDR_WIDTH = 4;
localparam MAX = 255;
parameter [7:0] DATA = 8'hFF;
```

Parser hỗ trợ:

- Arithmetic expressions trong parameter values
- Multiple parameter declarations
- Parameter resolution

4.4. Expression Parsing

4.4.1. Recursive Descent Parsing

Parser sử dụng **recursive descent parsing** để parse nested expressions:

Algorithm:

```
def parse_complex_expression(expr, output_signal):
    # Remove outer parentheses
    expr = remove_outer_parens(expr)

    # Find main operator (lowest precedence)
    main_op, pos = find_main_operator(expr)

    if main_op is None:
        return create_simple_assignment(output_signal, expr)

    # Split by main operator
    left_expr = expr[:pos]
    right_expr = expr[pos+len(main_op):]
```

```
    # Recursively parse left and right
    left_node = parse_sub_expression(left_expr)
    right_node = parse_sub_expression(right_expr)

    # Create node for main operator
    op_node = create_operation_node(main_op, [left_node, right_node])
    return op_node
```

4.4.2. Operator Precedence

Precedence (thấp → cao):

```
1. Logical OR (||)
2. Bitwise OR (|)
3. XOR (^)
4. Bitwise AND (&)
5. Logical AND (&&)
```

Ví dụ: `a | b & c` được parse thành `a | (b & c)` vì `&` có precedence cao hơn `|`.

4.4.3. Parentheses Matching

Parser xử lý parentheses đúng cách:

- Count parentheses depth để tìm operators ngoài cùng
- Match opening/closing parentheses
- Remove outer parentheses khi parse sub-expressions

4.4.4. Nested NOT Expressions

Với nested expressions như `~(a & b)`:

1. **Detection**: Phát hiện `~` ở đầu expression
2. **Operand extraction**: Lấy `(a & b)`
3. **Nested parsing**: Recursively parse `(a & b)` → tạo AND node
4. **NOT application**: Apply NOT lên kết quả AND node

Implementation:

```python
def parse_not_operation(lhs, rhs):
    operand = rhs.replace('~', '').strip()

    if operand.startswith('(') and operand.endswith(')'):
        # Nested expression
        nested_expr = operand[1:-1].strip()
        temp_signal = create_temp_signal()

        # Parse nested expression
        parse_complex_expression(temp_signal, nested_expr)

        # Create NOT node with temp_signal as input
        create_not_node(lhs, temp_signal)
    else:
        # Simple NOT
        create_not_node(lhs, operand)
```

4.5. Supported Constructs

4.5.1. Assign Statements

```
assign output = expression;
assign out = a & b;
assign result = (a | b) & c;
```

4.5.2. Always Blocks

Combinational:

```
always @(*) begin
    out = a & b;
end
```

Sequential:

```
always @(posedge clk) begin
    q <= d;
end
```

4.5.3. Case Statements

```
case (sel)
    2'b00: out = in0;
    2'b01: out = in1;
    2'b10: out = in2;
    default: out = 0;
endcase
```

Parser convert case statements thành MUX trees.

4.5.4. Generate Blocks

```
generate
    for (genvar i = 0; i < 8; i++) begin
        assign out[i] = in[i] & enable;
    end
endgenerate
```

Parser unroll generate blocks.

4.5.5. Bit Manipulation

- **Bit slices**: `signal[7:0]`, `signal[3]`
- **Concatenation**: `{a, b, c}`
- **Replication**: `{8{1'b1}}`

4.5.6. Module Instantiation

Named ports:

```
adder u1 (.a(a_in), .b(b_in), .sum(sum_out));
```

Ordered ports:

```
adder u1 (a_in, b_in, sum_out);
```

4.6. AST (Abstract Syntax Tree)

4.6.1. AST Structure

AST được biểu diễn internally trong quá trình parsing, sau đó được convert sang Netlist Dictionary.

4.6.2. AST Dump Utility

MyLogic cung cấp utility để dump AST structure (tương tự Yosys `-dump_ast`):

```
NETLIST_MODULE <module_name>
  NETLIST_WIRE <signal> input port
  NETLIST_WIRE <signal> output port

  NETLIST_ASSIGN <output>
    NETLIST_BIT_OR <node_id>
      NETLIST_BIT_OR <node_id>
        NETLIST_IDENTIFIER <signal>
        NETLIST_IDENTIFIER <signal>
      NETLIST_IDENTIFIER <signal>
```

Usage:

```
# CLI
mylogic> dump_ast

# Standalone
python tools/dump_ast.py file.v
```

PHẦN 5: SYNTHESIS ENGINE - NETLIST TO AIG CONVERSION

5.1. Tổng Quan Synthesis

5.1.1. Mục Đích

Synthesis là bước chuyển đổi Netlist Dictionary sang And-Inverter Graph (AIG). Đây là bước **technology-independent**, chỉ làm chuyển đổi representation, không tối ưu hóa.

5.1.2. Phân Biệt Synthesis vs Optimization

- **Synthesis**: Chuyển đổi representation (Netlist → AIG), **không tối ưu hóa**
- **Optimization**: Tối ưu hóa AIG về area/delay/power

5.2. AIG Data Structure

5.2.1. AIG Class

```
class AIG:
    def __init__(self):
        self.nodes: Dict[int, AIGNode] = {}
        self.next_node_id = 0

        # Constants
        self.const0 = self._create_node('CONST0')
        self.const1 = self._create_node('CONST1')

        # Primary Inputs
        self.pis: Dict[str, AIGNode] = {}

        # Primary Outputs
        self.pos: List[Tuple[AIGNode, bool]] = []

        # Structural hashing
        self.hash_table: Dict[Tuple[int, int, bool, bool], int] = {}
```

5.2.2. AIGNode Structure

```python
class AIGNode:
    def __init__(self, node_id, node_type, left=None, right=None,
                 left_inverted=False, right_inverted=False, var_name=None):
        self.node_id = node_id
        self.node_type = node_type  # CONST0, CONST1, PI, AND
        self.left = left
        self.right = right
        self.left_inverted = left_inverted
        self.right_inverted = right_inverted
        self.var_name = var_name  # For PI nodes
        self.level = 0  # Logic level
```

5.3. Gate Conversion Algorithms

5.3.1. AND Gate

```
AND(a, b) = create_and(a, b)  # Giữ nguyên
```

5.3.2. OR Gate (De Morgan's Law)

```
OR(a, b) = NOT(AND(NOT(a), NOT(b)))
         = create_not(create_and(create_not(a), create_not(b)))
```

Formula: `a OR b = !(!a AND !b)`

5.3.3. XOR Gate

```
XOR(a, b) = OR(AND(NOT(a), b), AND(a, NOT(b)))
          = create_or(
              create_and(create_not(a), b),
              create_and(a, create_not(b))
            )
```

Formula: `a XOR b = (!a AND b) OR (a AND !b)`

5.3.4. NAND Gate

```
NAND(a, b) = NOT(AND(a, b))
           = create_not(create_and(a, b))
```

5.3.5. NOR Gate

```
NOR(a, b) = AND(NOT(a), NOT(b))
          = create_and(create_not(a), create_not(b))
```

Note: Không cần NOT cuối vì đã có NOT ở inputs.

5.3.6. XNOR Gate

```
XNOR(a, b) = NOT(XOR(a, b))
           = create_not(create_xor(a, b))
```

5.3.7. NOT Gate

```
NOT(a) = AND(a, const1) với left_inverted=True
```

Trong AIG, NOT được biểu diễn bằng inversion flags.

5.4. Conversion Algorithm

5.4.1. Algorithm Chi Tiết

```
Algorithm: Convert Netlist → AIG

1. Initialize AIG
   - Create CONST0, CONST1 nodes
```

```
    - Initialize hash table

2. Create Primary Inputs (PI)
   - For each input signal:
     - Create PI node
     - Store in signal_mapping

3. Create Constants
   - Handle constant signals (const_True, const_False, 1'b1, 1'b0)

4. Convert Nodes (Topological Order)
   For each node in netlist:
     a. Get input signals
     b. Convert to AIG nodes (from signal_mapping or create if needed)
     c. Apply gate type conversion:
        - AND: create_and()
        - OR: create_or() → NOT(AND(NOT(a), NOT(b)))
        - XOR: create_xor() → OR(AND(!a,b), AND(a,!b))
        - NAND: create_and() + create_not()
        - NOR: AND(NOT(a), NOT(b))
        - NOT: create_not()
     d. Store result in signal_mapping

5. Create Primary Outputs (PO)
   - For each output signal:
     - Get AIG node from signal_mapping
     - Add to AIG.pos
```

5.4.2. Signal Mapping

Trong quá trình conversion, cần maintain mappings:

- **signal_mapping**: `signal_name → AIGNode` - Map signal names to AIG nodes
- **node_mapping**: `netlist_node_id → AIGNode` - Map netlist node IDs to AIG nodes
- **multibit_signal_mapping**: `signal_name → MultiBitAIGNode` - For multi-bit signals

5.4.3. Topological Ordering

Nodes được convert theo **topological order** để đảm bảo:

- Input nodes được convert trước output nodes
- Dependencies được resolve đúng

5.5. Multi-bit Operations

5.5.1. MultiBitAIGNode

```
class MultiBitAIGNode:
    def __init__(self, width: int, bits: List[AIGNode]):
        self.width = width
        self.bits = bits  # [bit0 (LSB), bit1, ..., bitN-1 (MSB)]
```

5.5.2. ADD Operation

Ripple-Carry Adder:

- Từng bit được add với carry từ bit trước
- Carry propagation từ LSB đến MSB
- Output: sum bits và carry out

Implementation:

```
def _convert_add_node(self, node_data):
    # Ripple-carry adder implementation
    # For each bit: sum = a XOR b XOR carry_in
    #               carry_out = (a AND b) OR (carry_in AND (a XOR b))
    # Build from LSB to MSB with carry propagation
```

5.5.3. SUB Operation

2's Complement Subtraction:

- `A - B = A + (~B) + 1`
- Sử dụng ripple-carry adder với inverted B và carry in = 1

5.5.4. MUX Operation

2-way MUX:

```
out = (sel ? in1 : in0)
    = (sel AND in1) OR (NOT(sel) AND in0)
```

N-way MUX:

- Build binary tree of 2-way MUXes
- Recursive implementation

5.5.5. EQ Operation

Equality Comparison:

- Compare từng bit position
- AND all bit comparisons
- Output: single-bit result

5.6. Structural Hashing

5.6.1. Hash Key

Mỗi AND node được hash bằng:

```
hash_key = (left_node_id, right_node_id, left_inverted, right_inverted)
```

Normalization: Đảm bảo `left_node_id <= right_node_id` để có canonical form.

5.6.2. Constant Folding

Trong quá trình tạo nodes, thực hiện **constant folding**:

- `AND(x, const0)` → `const0`
- `AND(x, const1)` → `x`
- `AND(x, !x)` → `const0` (tautology)

5.6.3. Duplicate Elimination

Khi tạo node mới:

1. Normalize inputs
2. Check hash table
3. Nếu đã tồn tại → return existing node
4. Nếu chưa tồn tại → create new node và store trong hash table

Lợi ích:

- Giảm số lượng nodes
- Automatic common subexpression elimination (CSE)
- Efficient memory usage

PHẦN 6: OPTIMIZATION ALGORITHMS

6.1. Tổng Quan Optimization

6.1.1. Mục Đích

Optimization nhằm tối ưu hóa AIG về:

- **Area**: Giảm số lượng gates
- **Delay**: Tối ưu hóa critical path
- **Power**: Giảm công suất tiêu thụ

6.1.2. Optimization Levels

MyLogic hỗ trợ 3 mức độ optimization:

1. **minimal**: Chỉ structural hashing cơ bản
2. **standard**: Các optimization algorithms cơ bản
3. **aggressive**: Tất cả optimization algorithms với nhiều iterations

6.2. Structural Hashing (Strash)

6.2.1. Định Nghĩa

Structural Hashing loại bỏ duplicate nodes bằng cách sử dụng hash table để store unique structures.

6.2.2. Implementation

- Hash table: `(left_id, right_id, left_inv, right_inv) → node_id`
- Normalization: `left_id <= right_id`
- Automatic duplicate elimination

6.2.3. Benefits

- Automatic CSE
- Memory efficiency
- Faster operations

6.3. Dead Code Elimination (DCE)

6.3.1. Định Nghĩa

Dead Code Elimination (DCE) loại bỏ logic không được sử dụng (unused logic).

6.3.2. Algorithm

1. Mark all Primary Outputs as "used"
2. Traverse backwards: Mark all nodes feeding into "used" nodes
3. Remove all unmarked nodes

6.3.3. Levels

- **basic**: Simple backward traversal
- **advanced**: Multiple passes với pattern matching
- **aggressive**: Aggressive elimination với more iterations

6.4. Common Subexpression Elimination (CSE)

6.4.1. Định Nghĩa

Common Subexpression Elimination (CSE) tìm và share redundant computations.

6.4.2. Algorithm

1. Identify common subexpressions
2. Create shared nodes
3. Replace all occurrences with shared node

6.4.3. Implementation

- Pattern matching
- Hash-based detection
- Replacement strategy

6.5. Constant Propagation (ConstProp)

6.5.1. Định Nghĩa

Constant Propagation propagate constant values through the circuit.

6.5.2. Algorithm

1. Identify constant nodes
2. Propagate constants forward

3. Simplify logic với constant inputs

6.5.3. Benefits

- Constant folding
- Logic simplification
- Dead code elimination opportunities

6.6. Logic Balancing (Balance)

6.6.1. Định Nghĩa

Logic Balancing balance logic depth để tối ưu hóa timing.

6.6.2. Algorithm

1. Calculate logic levels
2. Identify unbalanced paths
3. Restructure logic tree
4. Balance depth

6.6.3. Benefits

- Improved timing
- Better delay characteristics
- More uniform logic depth

PHẦN 7: TECHNOLOGY MAPPING

7.1. Tổng Quan Technology Mapping

7.1.1. Mục Đích

Technology Mapping map AIG sang target technology library (ASIC cells hoặc FPGA LUTs).

7.1.2. Strategies

- **area_optimal**: Tối ưu hóa về diện tích
- **delay_optimal**: Tối ưu hóa về delay
- **balanced**: Cân bằng giữa area và delay

7.2. Technology Libraries

7.2.1. ASIC Libraries

- Liberty format (`.lib`)
- JSON format (`.json`)
- Standard cells: AND, OR, NOT, NAND, NOR, etc.

7.2.2. FPGA Libraries

- LUT-based libraries
- Vendor-specific: Xilinx, Intel/Altera, Lattice, etc.

7.3. Mapping Algorithm

7.3.1. Pattern Matching

- Match AIG patterns với library cells
- Cost calculation (area, delay)
- Selection strategy

7.3.2. Output Generation

- Mapped netlist

- Verilog generation
- Statistics và reporting

PHẦN 8: VERIFICATION

8.1. Tổng Quan Verification

8.1.1. Mục Đích

Verification đảm bảo tính đúng đắn của synthesis và optimization bằng cách so sánh kết quả simulation.

8.1.2. Verification Types

1. **Synthesis Verification**: Original vs Synthesized
2. **Optimization Verification**: Synthesized vs Optimized

8.2. Verification Methodology

8.2.1. Industry Standard

Functional Simulation Comparison:

- Pre-synthesis vs Post-synthesis simulation
- Post-synthesis vs Post-optimization simulation

8.2.2. Test Vector Generation

- Exhaustive test vectors (all input combinations)
- Random test vectors
- Custom test vectors

8.3. ModelSim Integration

8.3.1. Integration Flow

```
1. Generate Verilog từ netlists
2. Create testbench
3. Run ModelSim simulation
4. Compare outputs
5. Report results
```

8.3.2. Verilog Generation

- Convert netlist sang Verilog
- Handle constants (const_True → 1'b1)
- Generate proper module structure

8.3.3. Testbench Generation

- Automatic testbench creation
- Apply test vectors
- Capture outputs
- Compare results

8.4. Verification Results

8.4.1. Result Format

```
{
    'passed': bool,
    'total_tests': int,
    'passed_tests': int,
    'failed_tests': int,
    'results': {
        'test_0': {'passed': True, 'inputs': {...}, 'outputs': {...}},
        ...
```

```
    }
}
```

8.4.2. Usage

```
# CLI
mylogic> complete_flow standard --verify

# Python
results = run_complete_flow(netlist, enable_verification=True,
test_vectors=vectors)
```

PHẦN 9: VLSI CAD ALGORITHMS

9.1. Binary Decision Diagrams (BDD)

9.1.1. Định Nghĩa

Binary Decision Diagrams (BDD) là một cấu trúc dữ liệu để biểu diễn Boolean functions một cách compact.

9.1.2. Applications

- Boolean function representation
- Equivalence checking
- Model checking

9.2. Boolean Expression Diagrams (BED)

9.2.1. Enhanced BDD

BED là enhancement của BDD với UP_ALL operation.

9.3. SAT Solver

9.3.1. Boolean Satisfiability

SAT Solver kiểm tra xem một Boolean formula có satisfiable không.

9.3.2. Applications

- Verification
- Test generation
- Optimization

9.4. Placement Algorithms

9.4.1. Random Placement

- Random placement of cells
- Fast nhưng không optimal

9.4.2. Force-Directed Placement

- Model cells như particles với forces
- Iterative optimization

9.4.3. Simulated Annealing

- Metaheuristic optimization
- Good quality results

9.5. Routing Algorithms

9.5.1. Maze Routing (Lee's Algorithm)

- Grid-based routing
- Guaranteed solution nếu có route tồn tại

9.5.2. Rip-up and Reroute

- Remove conflicting routes
- Re-route with different paths

9.6. Static Timing Analysis (STA)

9.6.1. Critical Path Analysis

- Find longest path
- Calculate delay

9.6.2. Slack Calculation

- Setup slack
- Hold slack
- Timing constraints

PHẦN 10: SIMULATION

10.1. Scalar Simulation

10.1.1. Single-bit Logic Gates

- Truth table evaluation
- Logic gate simulation

10.2. Vector Simulation

10.2.1. Multi-bit Operations

- Arithmetic operations
- Bitwise operations
- Vector handling

10.3. Auto-detection

- Automatic scalar vs vector mode detection
- Seamless switching

PHẦN 11: CLI INTERFACE

11.1. Interactive Shell

11.1.1. Features

- 36 commands (bao gồm file operations, synthesis, optimization, verification, VLSI CAD)
- Auto-completion
- Command history
- Real-time feedback

11.1.2. Commands

File Operations:

- `read <file>`: Load Verilog file
- `export [file]`: Export netlist to JSON

Analysis:

- `stats`: Circuit statistics
- `nodes`: Detailed node information
- `wires`: Wire analysis
- `dump_ast`: Dump AST structure

Synthesis:

- `synthesis [level]`: Run synthesis
- `strash`: Structural hashing
- `dce [level]`: Dead code elimination
- `cse`: Common subexpression elimination
- `constprop`: Constant propagation
- `balance`: Logic balancing

Complete Flow:

- `complete_flow [level] [strategy] [library] [--verify]`: Complete flow

VLSI CAD:

- `bdd`: Binary Decision Diagrams
- `sat`: SAT Solver
- `place [algorithm]`: Placement
- `route [algorithm]`: Routing
- `timing`: Static Timing Analysis

PHẦN 12: IMPLEMENTATION DETAILS

12.1. Programming Language và Tools

- **Language**: Python 3.8+
- **Libraries**: NumPy, Matplotlib
- **Optional**: Yosys, ModelSim, Graphviz

12.2. Code Organization

- Modular structure
- Clear separation of concerns
- Comprehensive documentation

12.3. Key Design Decisions

- AIG as canonical form
- Structural hashing for efficiency
- Modular parser architecture

PHẦN 13: KẾT QUẢ VÀ ĐÁNH GIÁ

13.1. Test Cases

13.1.1. Test Suite

- Basic gates (AND, OR, XOR)
- Bitwise operations
- Complex expressions
- Nested operations

13.1.2. Test Results

- All test cases pass
- Correct synthesis
- Proper optimization
- Successful verification

13.2. Limitations

13.2.1. Current Limitations

- Sequential logic chưa được hỗ trợ đầy đủ
- Memory arrays cần special handling
- Complex always blocks cần improvement

13.2.2. Workarounds

- Use combinational-only designs
- Extract combinational parts
- Use simpler test cases

13.3. Comparison với Yosys

- Feature comparison
- AST comparison
- Synthesis quality comparison

PHẦN 14: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

14.1. Kết Luận

14.1.1. Tổng Kết

MyLogic EDA Tool đã thành công trong việc:

- Phát triển Verilog parser đầy đủ tính năng
- Xây dựng synthesis engine chuyển đổi Netlist → AIG
- Implement các thuật toán optimization quan trọng
- Tích hợp verification system
- Phát triển VLSI CAD algorithms cơ bản

14.1.2. Đóng Góp

- Educational tool cho học tập và nghiên cứu
- Code dễ đọc và hiểu
- Comprehensive documentation
- Modular design dễ mở rộng

14.2. Hạn Chế

- Sequential logic chưa hỗ trợ đầy đủ
- Optimization chưa mạnh bằng tools chuyên nghiệp
- Performance cần được cải thiện

14.3. Hướng Phát Triển

14.3.1. Short-term

- Sequential logic support
- Memory support
- Performance improvements

14.3.2. Long-term

- Advanced optimization algorithms
- Parallel processing
- GUI interface
- Integration với other tools

PHẦN 15: TÀI LIỆU THAM KHẢO

15.1. Papers

1. A. Mishchenko et al., "ABC: A System for Sequential Synthesis and Verification"

2. R. K. Brayton and A. Mishchenko, "ABC: An Academic Industrial-Strength Verification Tool"
3. E. Sentovich et al., "SIS: A System for Sequential Circuit Synthesis"

15.2. Books

1. S. Devadas, A. R. Newton, "Algorithms for Hardware Design and Two Level Logic Synthesis"
2. G. De Micheli, "Synthesis and Optimization of Digital Circuits"

15.3. Standards

1. IEEE Std 1364-2005, "IEEE Standard for Verilog Hardware Description Language"

15.4. Tools và Documentation

1. Yosys Open Synthesis Suite Documentation
2. ABC Documentation
3. ModelSim User Guide

PHỤ LỤC

Phụ Lục A: Code Examples

A.1. Example Verilog File

```
module simple_and(
    input a, b,
    output out
);
```

```
    assign out = a & b;
endmodule
```

A.2. Usage Example

```
from parsers import parse_verilog
from core.complete_flow import run_complete_flow

# Parse
netlist = parse_verilog("examples/simple_and.v")

# Complete flow với verification
results = run_complete_flow(
    netlist,
    optimization_level="standard",
    enable_verification=True
)
```

Phụ Lục B: Command Reference

Xem CLI help: `mylogic> help`

Phụ Lục C: API Documentation

Xem source code comments và docstrings.

Kết thúc báo cáo